

The library used for data cleaning

```
1 # Library
2 library(tidyverse)
3 library(dplyr)
4 library(haven)
5 library(tibble)
6
7 #Making plot
8 library(ggplot2)
9
10 #imputations
11 library(mice)
12
13 #glmnet and VIF
14 library(glmnet)
15 library(car)
16
17 #rexposomes
18 library(rexposome)
19 library(Biobase)
20 set.seed(18)
```

1 Data

Need to be mindful:

```
1 # Import variables
2 exposure_data <- read.csv("/Volumes/Exposome_BIS-I4068/2. Data/10. Clean
  data/Variable/final_variable_vs5.csv")
3 # Import out data
4 out_data <- read.csv("/Users/victoriasjalimcopy/Documents/BIOX7011/Coding
  /BIOX7011/Final/clean_outcome_m12_vs3.csv", row.names = 1)
```

2 Inspect the data

We further reduced the variable size due to redundancy or not really important to be included in the analysis. We also combined some variables.

```
1 m#Excluding non-important variables
2 exposure_data <- exposure_data %>%
3   select(-c(mpht_t1t2, tothomesibs_t1t2_d, labdurtotmins_d, hresides_t1t2
4     ,
5     livestock, pctlile_bweight_aus_d, numsib,
6     loneparent24m_d, starts_with("dist_any"), smkrmhrs, loneparent
7     _t1t2_d, mpmarr_t1t2_d, smkroomdur_12m, mrecdrug12m, precdrug12m))
8 # Mutate redundant variables
9 red_vars <- exposure_data %>%
10   mutate(totpregsmoke = pregsrocket1_t1t2 + pregsrocket2_t1t2) %>%
11   mutate(totpregsmoke = case_when(totpregsmoke > 0 ~ 1,
12     totpregsmoke == 0 ~ 0)) %>%
13   mutate(passivesmoke = passive_T2 + passive_T3) %>%
14   mutate(fever = mpfevet1_t1t2 + mpfevet2_t1t2 + mpfevet3) %>%
```

```

14 mutate(antibiotic = antibiotic_T1_d + antibiotic_T2_d + antibiotic_T3_d
15 ) %>%
16 mutate(decor = ceybed + celivroom + cekitch + cebabyb) %>%
17 mutate(pregalc = rowMeans(cbind(pregalct1_tlt2, pregalct2_tlt2)))
18 #Remove the redundant variables
19 red_vars <- red_vars %>%
20 select(-c(pregsmokerm_tlt2, pregsmoket2_tlt2, passive_T3, passive_T2,
21 pregsmoket1_tlt2, mpfevet3, mpfevet1_tlt2, mpfevet2_tlt2, antibiotic_
22 T3_d, antibiotic_T1_d, antibiotic_T2_d, ceybed, celivroom, cekitch,
23 lsumDEP_DBPs_DEHP_tod_di, pregalct1_tlt2, pregalct2_tlt2, ends_with("
24 byb")))

```

2.1 Change the variable names

Change the variables names so it would be nicer and prettier.

```

1 #Change the colnames - so the variable names is nicer
2 colnames(red_vars) <- colnames(red_vars) %>%
3   gsub("_di$", "", .) %>%
4   gsub("_tlt2$", "", .) %>%
5   gsub("_m12$", "12m", .) %>%
6   gsub("hm$", "", .)
7
8 #Renaming
9 name_map <- c("l_8ohdg_w36_adj" = "l8ohdg_w36",
10 "l_8ohg_w36_adj" = "l8ohg_w36",
11 "ox_stress_gpfs" = "gPFSox",
12 "pss_score_301" = "pss_m1",
13 "pss_score_312" = "pss_m12",
14 "msmkno_12m" = "msmoke_m12",
15 "ssmkno_12m" = "psmoke_m12",
16 "pparchilddied" = "parentchildied",
17 "hhincome12m_d" = "hhincome12m",
18 "fcobaus_d" = "fcobaus",
19 "mcobaus_d" = "mcobaus",
20 "modebirth_new" = "modebirth",
21 "pc1_modern_healthy" = "healthy_diet",
22 "pc2_western_unhealthy" = "unhealthy_diet",
23 "pc3_traditional_anglo" = "anglo_diet",
24 "pred_pm25_satnowood12m" = "pm25_12m",
25 "pred_pm25_satnowood_a" = "pm25_a",
26 "mpnonpropancontact" = "animalcontact",
27 "dist_maj_rd_m_a" = "roaddist_a",
28 "pred_no212m" = "no2_12m",
29 "pred_no2_a" = "no2_a",
30 "dist_maj_rd_m12m" = "roaddist_12m",
31 "pnt" = "paint",
32 "chl" = "chlorine",
33 "can" = "candle",
34 "sc" = "scents",
35 "aero" = "aerosol",
36 "airfr" = "airfresh",
37 "hmalcoprob" = "alcoprob",
38 "hmdrugprob" = "drugprob",
39 "mppetsany" = "anypets",

```

```

40 "zscore_bweight_aus_d" = "zscore_bweight",
41 "maleYN" = "sex")
42
43 red_vars <- red_vars %>%
44   rename_with(~ name_map[.x], .cols = names(name_map))
45
46 cov <- red_vars %>%
47   select(bisid, sex, zscore_bweight)
48
49 red_vars <- red_vars %>%
50   select(-sex, -zscore_bweight)

```

2.2 Modifications

Modified father employment since everything that <0 usually don't know or refused to answer that is what it better to leave it as NA.

```

1 summary(red_vars)
2
3 # Modify first
4 red_vars$fempl[red_vars$fempl < 0] <- NA
5 summary(red_vars)

```

2.3 Plot missingness

```

1 missing_val <- red_vars1 %>%
2   pivot_longer(everything(), names_to = "exposures", values_to = "value")
3   %>%
4   group_by(exposures) %>%
5   summarise(avg = mean(value, na.rm=T),
6             sd = sd(value, na.rm = T),
7             na = sum(is.na(value)),
8             percent_na = (na/697)*100)
9
10 # Statistics for missing value
11 missing_plot <- red_vars1 %>%
12   pivot_longer(everything(), names_to = "exposures", values_to = "value")
13   %>%
14   group_by(exposures) %>%
15   summarise(avg = mean(value, na.rm=T),
16             sd = sd(value, na.rm = T),
17             na = sum(is.na(value)),
18             percent_na = (na/697)*100) %>%
19   mutate(exposures = reorder(exposures, na)) %>%
20   ggplot(aes(x = percent_na, y=exposures, fill = "blue")) +
21   geom_col() +
22   theme_minimal() +
23   theme(
24     legend.position = "none",
25     axis.text.y = element_text(size = 7),
26     panel.grid.major.x = element_line(colour = "grey85")
27   ) +
28   labs(x = "Percent NA (%)", y = "Exposures") +
29   xlim(0, 100)

```

```
28
29 missing_plot
```

3 Imputations

```
1 #Prepare exposure data excluding the bisid
2 exp_dat <- red_vars[, -which(names(red_vars) == "bisid")]
3
4 #Create the prediction matrix for imputations
5 #see tamayo uria et al
6 pred_matrix <- quickpred(exp_dat,
7                           mincor = 0.1,
8                           minpuc = 0.4)
9 result <- try({
10   mice(exp_dat,
11         method = "pmm",
12         pred = pred_matrix,
13         m = 5,
14         seed = 17550,
15         printFlag = FALSE)
16 }, silent = TRUE)
17
18 result_exp <- complete(result,1)
19
20 sum(is.na(result_exp)) #0
21
22 # Compare the distributions between two datasets
23 library(ggforce)
24 bind_rows(
25   red_vars[-1] %>% mutate(.dataset = "A"),
26   result_exp %>% mutate(.dataset = "B")
27 ) %>%
28   pivot_longer(where(is.numeric), names_to = "var", values_to = "val")
29   %>%
30   group_by(.dataset, var) %>%
31   ggplot(aes(val, colour = .dataset, fill = .dataset)) +
32   geom_density(alpha = 0.25, na.rm = TRUE, adjust = 1) +
33   facet_wrap(~ var, nrow = 10, ncol = 10, scales = "free") +
34   labs(x = NULL, y = "Density", colour = "Dataset", fill = "Dataset") +
35   theme_classic(base_size = 10)
```

The analysis are robust as there is not much significant different before and after imputations.

3.1 Save the data

```
1 # writing the exposures
2 finalv_imp5 <- cbind(bisid = red_vars$bisid, result_exp)
3
4 write.csv(finalv_imp5, "fix_final_vars_148.csv", row.names = F)
5
6 # The output
7 finalout <- out_data %>%
8   rename(sLRTI = cumwhecgh) %>%
9   select(-"cumwhecgh_cat")
```

```

10
11 # Merge it with covariates
12 finalout <- merge(finalout, cov, by="bisid")
13
14 # Write the outcomes
15 write.csv(finalout, "out_imp_cov_vs.csv", row.names = F)

```

4 Data analysis

In here we start using rexposome package.

There are 3 requirements for rexposome data:

- Exposures: all the exposures every rows representing all the participants, the columns is all the exposures. First columns must be the ID.
- Outcome/phenotypes: all the outcomes and its covariates. First columns must be the ID.
- Description: all information about the exposures. The rows is the exposures, make sure you have the columns family to know which family the exposures belongs to.

Bare with me, that there are few trials that I was doing before finalising to the final models. The code would be the same. I decided to use birth weight and sex as covariates from a priori and this covariates would be included in the outcome data.

```

1 # Importing all the data
2 #Variable
3 finalvars <- read.csv("/Users/victoriasjalimcopy/Documents/BIOX7011/
  Coding/BIOX7011/Final/fix_final_vars_148.csv")
4
5 #Description
6 desc_new <- read.csv("/Users/victoriasjalimcopy/Documents/BIOX7011/Coding
  /BIOX7011/Final/description_rename.csv",header = T)
7
8 # Outcome
9 out_data <- read.csv("/Users/victoriasjalimcopy/Documents/BIOX7011/Coding
  /BIOX7011/Final/out_imp_cov_vs.csv")

```

We tried by creating the exposome first.

```

1 eset_imp <- loadExposome(
2   exposures = finalvars,
3   phenotype = out_data,
4   description = desc_new,
5   description.famCol = "Family"
6 )
7
8 eset_imp
9
10 #See the missing data
11 plotMissings(eset_imp, set = "exposures") # For exposures

```

At first, I thought that rexposome package automatically using loadexposome would automatically define the ID. However, the bisid was included in the analysis. After read the manuals, turns out you need to make a path for each files and use the specific parameters to define the ID column (this parameter were not there in the loadexposome). Therefore, we are using the readExposome.

```

1 #Writing the path
2 final_vars_path <- "/Users/victoriasjalimcopy/Documents/BIOX7011/Coding/
  BIOX7011/Final/fix_final_vars_148.csv"
3 final_out_path <- "/Users/victoriasjalimcopy/Documents/BIOX7011/Coding/
  BIOX7011/Final/out_imp_cov_vs.csv"
4 final_desc_path <- "/Users/victoriasjalimcopy/Documents/BIOX7011/Coding/
  BIOX7011/Final/description_rename.csv"
5
6 #using the rexposome
7 eset_final <- readExposome(exposures = final_vars_path,
8                             phenotype = final_out_path,
9                             description = final_desc_path,
10                             description.famCol = "Family",
11                             description.expCol = "Name",
12                             exposures.samCol = "bisid", # Define the col
13                             for bisid
14                             phenotype.samCol = "bisid") # Define the col
15 # 85 exposures
16 plotFamily(eset_final, family = "all")

```

4.1 Transformations

At first, I want to use box-cox transformations. However, it kept showing me an error and this is because we do have some negative value such as for PC1 for diets and phtalates. After few research, that is why we are doing yeo-johnson transformations, the extension of box-cox that can handle negative values.

```

1 library(e1071)
2 library(car)
3
4 # Function for the transformation
5 expo_best_trans <- function(var) {
6
7   transformed_df <- var
8
9   for (x in colnames(var)) {
10     if (is.numeric(var[[x]])) {
11       yj <- powerTransform(var[[x]], family = "yjPower")
12       transformed_df[[x]] <- yjPower(var[[x]], lambda = yj$lambda)
13     }
14   }
15   return(transformed_df)
16 }
17
18 # Transform the continuous data
19 eset_imp_trans <- trans(eset_final, fun = expo_best_trans)
20 eset_imp_trans
21
22 #Standardize
23 eset_imp_t_s <- standardize(eset_imp_trans, method = "normal")
24 eset_imp_t_s
25 plotFamily(eset_imp_t_s, family = "all")

```



```

6         exposures.samCol = "bisid",
7         phenotype.samCol = "bisid",
8         exposures.asFactor = 1) # Included all data
9         that at least have one factor
10 eset_final
11 plotFamily(eset_final, family = "all")

```

5.1 Pre-processing

Same pre-processing done from before.

```

1 # Transform the continuous data
2 eset_imp_trans <- trans(eset_final, fun = expo_best_trans)
3 eset_imp_trans
4
5 #Standardize
6 eset_imp_t_s <- standardize(eset_imp_trans, method = "normal")
7 eset_imp_t_s
8 plotFamily(eset_imp_t_s, family = "all")

```

5.2 ExWAS

```

1 #Look at the phenotype
2 phenotypeNames(eset_imp_t_s)
3
4 #for the cumwheeze
5 ex_cum <- exwas(eset_imp_t_s, formula = sLRTI ~ sex + zscore_bweight,
6                 family = "gaussian")
7 ex_cum
8
9 plotExwas(ex_cum)
10 plotEffect(ex_cum)

```

5.3 InExWAS

```

1 inv_ex <- invExWAS(eset_imp_t_s, formula = ~sex + zscore_bweight)
2 #head(extract(inv_ex))
3
4 # The visualisation
5 clr <- rainbow(length(familyNames(eset_imp_t_s)))
6 names(clr) <- familyNames(eset_imp_t_s)
7 plotExwas(inv_ex, color = clr)

```

The curse of dimensionality happened in here. In rexposome, they used the method tef or threshold effective test, where we create our own p-value depended on the size of the data. Increase number of exposure will lead to smaller p-value. When testing many univariate tests (e.g., testing hundreds of exposures for association with one health outcome), the chance of getting false positives increases. Therefore, we didn't find a significant result despite probably some of the variables actually have a low p-value, because didn't passed the tef (The TEF was 7.74e-04)

That is why we further separate the analysis into postnatal and prenatal.

6 Dictionary

To have more clarity on our graph, however, there should be no spaces in the variables name, we will make dictionary that corresponds to the variables name.

```
1 ## Dictionary
2
3 dict_name <- c(
4   pregalc = "Prenatal Alcohol Intake",
5   no2_12m = "NO2 (12m)",
6   no2_a = "NO2 (pregnancy)",
7   pm25_12m = "PM2.5 (12m)",
8   pm25_a = "PM2.5 (pregnancy)",
9   roaddist_12m = "Road distance (12m)",
10  roaddist_a = "Road distance (pregnancy)",
11  atregl2m = "aTregs (12m)",
12  GlycA_a = "GlycA (pregnancy)",
13  GlycA_b = "GlycA (birth)",
14  GlycA12m = "GlycA (12m)",
15  gPFSox = "gPFSox",
16  l8ohdg_w36 = "8-OH-dG",
17  l8ohg_w36 = "8-OH-Gua",
18  rtregl2m = "rTregs (12m)",
19  bfany = "Any Breastfed",
20  healthy_diet = "Healthy Diet",
21  unhealthy_diet = "Unhealthy Diet",
22  anglo_diet = "Anglo Diet",
23  alcoprob = "Alcohol problem",
24  drugprob = "Drugs problem",
25  eds_score = "EDS Score",
26  financialprob = "Financial problem",
27  malcot12m = "Mother alcohol intake (12m)",
28  palcot12m = "Father alcohol intake (12m)",
29  parentchildied = "Any parents/child died",
30  policecourt = "Police/legal problem",
31  pss_m1 = "PSS (1m)",
32  pss_m12 = "PSS (12m)",
33  reldied = "Relative died",
34  stolenva1 = "Robbery",
35  animalcontact = "Animal contact (not house)",
36  antibiotic = "Pregnancy antibiotic used",
37  anypets = "Pets",
38  disinfhmt1 = "Disinfectant (pregnancy)",
39  fever = "Fever (pregnancy)",
40  hresidesadult = "Number of Adults",
41  hresideschild = "Number of children",
42  modebirth = "Mode of birth",
43  mpfarm = "Farm work",
44  buf_500 = "Green space",
45  NDVI = "NDVI",
46  aerosol = "Aerosol",
47  airfresh = "Air fresheners",
48  candle = "Candle",
49  carpcl = "Carpet cleaner",
50  chlorine = "Chlorine",
51  decor = "Decoration in the house",
52  disinf = "Disinfectant",
```

```

53 fabsofbabycl = "Fabric softener",
54 fumany = "Any fumigations",
55 glasscl = "Glass cleaner",
56 laubabycl = "Liquid laundry",
57 mcoil = "Mosquito coil",
58 paint = "Paint",
59 pest = "Pest",
60 repel = "Repellent",
61 scents = "Scented product",
62 stainbabycl = "Stain remover",
63 turp = "Turpentine",
64 shannon_a = "Shannon index (pregnancy)",
65 shannon12m = "Shannon index (12m)",
66 fcobaus = "Father born in Australia",
67 feducation = "Father education",
68 fempl = "Father employment",
69 hhincome12m = "Household income",
70 mage = "Mother Age",
71 mcobaus = "Mother born in Australia",
72 meducation = "Mother education",
73 mempl = "Mother employment",
74 seifa_disad = "SEIFA disadvantages score",
75 msmoke_m12 = "Mother smoke (12m)",
76 passivesmoke = "Passive smoke (pregnancy)",
77 psmoke_m12 = "Father smoke (12m)",
78 totpregsmoke = "Mother smoke (pregnancy)",
79 zscore_bweight = "Birthweight",
80 cumwhecgh = "sLRTI"
81 )
82
83 pretty_label <- function(x) {
84   ifelse(x %in% names(dict_name), dict_name[x], x)}
85
86 # I ask chat gpt for more colors because IDK what other colors R have
87 exposure_colors <- c(
88   "Family adversity" = "#FF4500",      # Orange red
89   "Biomarker" = "#B8860B",             # Crimson
90   "Diet" = "#8B4513",                  # Saddle brown
91   "SEP" = "#000080",                   # Navy blue
92   "Smoking" = "hotpink",                # Indigo
93   "Ambient air quality" = "#800080",    # Purple
94   "MGM" = "orange",                    # Deep pink
95   "Indoor air quality " = "darkgrey",
96   "Green space" = "green",              # Dark green
97   "FEME" = "#008B8B",                  # Dark cyan
98   "Phenols" = "darkgreen",             # Dark olive green
99   "Alcohol intake" = "#DC143C",        # Dark goldenrod
100  "Phtalates " = "red"
101 )

```

7 Postnatal

```

1 #Updated the description
2 desc_new <- read.csv("description_rename.csv")

```

```

3
4 #Select the postnatal variable
5 vars_post <- desc_new %>%
6   filter(Status == "postnatal") %>%
7   pull(Name)
8 vars_post <- c("bisid", vars_post)
9
10 #Write the variables documents
11 post_vars <- finalvars[,vars_post]
12 post_vars[] <- lapply(post_vars, as.numeric)
13 write.csv(post_vars, "exposome_post_vars_vs.csv", row.names = F)
14
15 #Description for postnatal
16 vars_post <- vars_post[vars_post != "bisid"]
17 desc_post <- desc_new[desc_new$Name %in% vars_post,]
18 write.csv(desc_post, "description_update_post.csv")
19
20 final_vars_post_path <- "/Users/victoriasjalimcopy/Documents/BIOX7011/
   Coding/BIOX7011/Final/exposome_post_vars_vs.csv"
21
22 final_desc_post_path <- "/Users/victoriasjalimcopy/Documents/BIOX7011/
   Coding/BIOX7011/Final/description_update_post.csv"
23
24 #using the rexposome
25 eset_post <- readExposome(
26   exposures = final_vars_post_path,
27   phenotype = final_out_path,
28   description = final_desc_post_path,
29   description.famCol = "Family",
30   description.expCol = "Name",
31   exposures.samCol = 1,
32   phenotype.samCol = 1,
33   exposures.asFactor = 1)
34 eset_post
35
36 # Transform the continuous data
37 eset_post_t <- trans(eset_post, fun = expo_best_trans)
38 eset_post_t
39
40 #Standardize
41 eset_post_ts <- standardize(eset_post_t, method = "normal")
42 eset_post_ts
43
44 # Summary Statistics
45 plotFamily(eset_post, family = "all")
46 plotFamily(eset_post_ts, family = "all")

```

7.1 Correlations

We are doing Pearson's correlations. If we wanted to changed the variable names, we need to make sure to changed multiple data frame, since mismatch of names will not produced a graph. To be mindful: assay data (correlation matrix) and feature data (contains family),

```

1 #Correlation
2 eset_post_c <- correlation(eset_post_ts)
3 eset_post_c

```

```

4
5 # Changing the row names so it will have pretty names
6 # Change the rownames for the feature data first
7 rownames(fData(eset_post_c)) <- dict_name[rownames(fData(eset_post_c))]
8 # Change the rownames for the the assay data
9 rownames(eset_post_c@assayData$corr) <- dict_name[rownames(eset_post_
  c@assayData$corr)]
10 # Also the colnames
11 colnames(eset_post_c@assayData$corr) <- dict_name[colnames(eset_post_
  c@assayData$corr)]
12
13 #Plot
14 plotCorrelation(eset_post_c, type = "circos")

```

7.2 PCA

```

1 #Trying with all T
2 pca_post <- pca(eset_post_ts, pca = T)
3 plotPCA(pca_post, set = "all")
4
5 plotPCA(pca_post, set = "exposures", show.exposures = FALSE) +
6   theme(legend.position = "right") +
7   theme_minimal()
8
9 #Plot the exposures
10 plotEXP(pca_post) +
11   # Change the scale
12   ggplot2::scale_fill_gradient2(low= "steelblue", mid = "white", high = "
    red") +
13   # Change the variables name
14   ggplot2::scale_y_discrete(labels = pretty_label) +
15   labs(x = "PCs", y = "Exposures")
16
17 #Plot the phenotype
18 plotPHE(pca_post) +
19   labs(x = "PCs", y = "Phenotype")

```

7.3 ExWAS

Actually I run it without changing the name first, then I included the rename all the variables again.

```

1 #for the cumwheeze
2 ex_cum_post <- exwas(eset_post_ts, formula = sLRTI ~ sex + zscore_bweight
  , family = "gaussian", warnings = F)
3 ex_cum_post
4 head(ex_cum_post@comparison)
5 #plotExwas(ex_cum_post) +
6   # geom_point(size=3) + scale_color_manual(values = exposure_colors)
7 #plotEffect(ex_cum_post)
8
9 exwas_post <- as.data.frame(ex_cum_post@comparison) %>%
10   mutate(label = dplyr::recode(rownames(ex_cum_post@comparison),
11     GlycA12m = "GlycA (12m)",
12     financialprob = "Financial Problem",

```

```

13         fumany = "Any fumigations",
14         ms smoke_m12 = "Mother smoke (12m)",
15         sig = pvalue < 0.05)
16
17 # Volcano plot
18 #plotVolcano(ex_cum_pre) +
19 ggplot(exwas_post, aes(x=as.numeric(effect), y= -log10(as.numeric(pvalue)
20     ),
21     colour = sig)) +
22     geom_point() +
23     labs(x = "Effect", y = expression(-log[10](p-value))) +
24     # Create the threshold p-value
25     geom_hline(yintercept = -log10(1.46e-03),
26         col="darkred", linetype = "dashed") +
27     geom_hline(yintercept = -log10(0.05),
28         col="darkgreen", linetype = "dashed") +
29     geom_vline(xintercept = 0, col="blue", linetype = "dashed") +
30     scale_colour_manual(values = c(`FALSE` = "black",
31         `TRUE` = "red")) +
32     coord_cartesian(xlim = c(-5, 5)) +
33     guides(color = "none", shape = "none") +
34     theme_minimal() +
35     theme(panel.border = element_rect(colour = "black",
36         linewidth = 0.8, fill = NA),
37         panel.background = element_rect(fill = "white",
38         color = NA),
39         plot.background = element_rect(fill = "white",
40         color = NA),
41         legend.position = "none") +
42     ggrepel::geom_text_repel(data = subset(exwas_post, sig),
43         aes(label = label),
44         size = 4,
45         box.padding = unit(0.35, "lines"),
46         point.padding = unit(0.35, "lines"),
47         max.overlaps = Inf,
48         segment.color = "black") +
49     coord_cartesian(xlim = c(-0.5, 0.5)) +
50     labs(x="Effect")
51 #write.csv(ex_cum_post@comparison, "Result_postnatal_exwas.csv")

```

7.4 InExWAS

```

1 inexas_cum <- invExWAS(eset_post_ts, formula = ~zscore_bweight + sex)
2 #head(extract(inexas_cum))
3 inexas_cum
4
5 # The visualisation
6 # This is the first visualisations but I don't like it
7 plotExwas(inexas_cum) +
8     geom_point(size=3) + scale_color_manual(values = exposure_colors)
9
10 #write.csv(inexas_cum@comparison, "Result_post_inexas.csv")
11
12 # Better visualisations

```

```

13 inxwas_post <- as.data.frame(inxwas_cum@comparison) %>%
14   mutate(label = dplyr::recode(rownames(inxwas_cum@comparison),
15     pss_m12 = "PSS (12m)",
16     financialprob = "Financial Problem",
17     stolental = "Robbery",
18     fumany = "Any fumigations",
19     msmoke_m12 = "Mother smoke (12m)",
20     buf_500 = "Green space",
21     pm25_12m = "PM2.5",
22     drugprob = "Drugs problem",
23     repel = "Repellent"),
24     sig = pvalue < 0.05)
25
26 # Volcano plot
27 #plotVolcano(ex_cum_pre) +
28 ggplot(inxwas_post, aes(x=as.numeric(effect), y= -log10(as.numeric(
29   pvalue)),
30   colour = sig)) +
31   geom_point() +
32   labs(x = "Effect", y = expression(-log[10](p-value))) +
33   # Create the threshold p-value
34   geom_hline(yintercept = -log10(1.46e-03),
35     col="darkred", linetype = "dashed") +
36   geom_hline(yintercept = -log10(0.05),
37     col="darkgreen", linetype = "dashed") +
38   geom_vline(xintercept = 0, col="blue", linetype = "dashed") +
39   scale_colour_manual(values = c(`FALSE` = "black",
40     `TRUE` = "red")) +
41   coord_cartesian(xlim = c(-0.25, 0.25)) +
42   guides(color = "none", shape = "none") +
43   theme_minimal() +
44   theme(panel.border = element_rect(colour = "black",
45     linewidth = 0.8, fill = NA),
46     panel.background = element_rect(fill = "white",
47     color = NA),
48     plot.background = element_rect(fill = "white",
49     color = NA),
50     legend.position = "none") +
51   ggrepel::geom_text_repel(data = subset(inxwas_post, sig),
52     aes(label = label),
53     size = 4,
54     box.padding = unit(0.35, "lines"),
55     point.padding = unit(0.35, "lines"),
56     max.overlaps = Inf,
57     segment.color = "black",
58     min.segment.length = 0) +
59   coord_cartesian(xlim = c(-0.5, 0.5)) +
60   labs(x="Effect")
61 #ggsave("Result/inxwas_volcano.png", v_inxwas_po)

```

7.5 MexWAS

We are using elastic net. Using lambda min and alpha 0.5, with 10 CV.

- If coefficient > 0: higher values of that variable are associated with an increased value of the

outcome

- If coefficient < 0: higher values of the predictor are associated with a decrease in the outcome.

```
1 # Get all the Exposures
2 post_vars_st <- expos(eset_post_ts)
3 # Obtain the outcome for postnatal
4 Y <- pData(eset_post_ts)
5 Y <- Y$cumwhecgh
6
7 # Do cross validations (to find the lambda)
8 cv_enet_post <- cv.glmnet(as.matrix(post_vars_st), Y, family = "gaussian"
9 , alpha = 0.5)
10 # Enet with selected lambda
11 enet_post <- glmnet(post_vars_st, Y, alpha = 0.5, lambda = cv_enet_post$
12 lambda.min)
13
14 # See which variables are important
15 coef(enet_post, s = cv_enet_post$lambda.min)
16
17 # Visualisations
18 coef(enet_post, s = cv_enet_post$lambda.min) %>%
19 as.matrix() %>% as.data.frame() %>%
20 setNames("coef") %>%
21 rownames_to_column("feature") %>%
22 filter(feature != "(Intercept)") %>%
23 ggplot(aes(x = feature, y = coef, fill = coef)) +
24 geom_col() +
25 scale_x_discrete(labels = pretty_label) +
26 scale_fill_gradient2(low = "steelblue", mid = "white", high = "red",
27 midpoint = 0) +
28 coord_flip() +
29 labs(x = NULL, y = "Coefficient", fill = "Value") +
30 theme_minimal(base_size = 12)
```

8 Prenatal

We are begin to analyze about prenatal exposures.

```
1 #Select the postnatal variable
2 vars_pre <- desc_new %>%
3 filter(Status == "prenatal") %>%
4 pull(Name)
5 vars_pre <- c("bisid", vars_pre)
6
7 #Write the variables documents
8 pre_vars <- finalvars[,vars_pre]
9 pre_vars[] <- lapply(pre_vars, as.numeric)
10 write.csv(pre_vars, "exposome_pre_vars_vs.csv", row.names = F)
11
12 #Description for prenatal
13 vars_pre <- vars_pre[vars_pre != "bisid"]
14 desc_pre <- desc_new[desc_new$Name %in% vars_pre,]
15 write.csv(desc_pre, "description_update_pre.csv")
16
```

```

17 final_vars_pre_path <- "/Users/victoriasjalimcopy/Documents/BIOX7011/
   Coding/BIOX7011/Final/exposome_pre_vars_vs.csv"
18
19 final_desc_pre_path <- "/Users/victoriasjalimcopy/Documents/BIOX7011/
   Coding/BIOX7011/Final/description_update_pre.csv"
20
21 #using the rexposome
22 eset_pre <- readExposome(
23   exposures = final_vars_pre_path,
24   phenotype = final_out_path,
25   description = final_desc_pre_path,
26   description.famCol = "Family",
27   description.expCol = "Name",
28   exposures.samCol = 1,
29   phenotype.samCol = 1,
30   exposures.asFactor = 1)
31 eset_pre
32
33 # Transform the continuous data
34 eset_pre_t <- trans(eset_pre, fun = expo_best_trans)
35 eset_pre_t
36
37 #Standardize
38 eset_pre_ts <- standardize(eset_pre_t, method = "normal")
39 eset_pre_ts

```

8.1 Correlations

Same as above in 7.1. However, in here, there is some variables (i.e., like the phenols), that wasn't in the dictionary, which cause an error. We need to do different way.

```

1 #Correlation
2 eset_pre_c <- correlation(eset_pre_ts)
3 eset_pre_c
4
5 # Only change the name if it's contain in the dictionary
6 if (!is.null(dict_name) && length(dict_name) > 0){
7   # For fData rownames
8   new_fdata_names <- rownames(fData(eset_pre_c))
9   idx <- new_fdata_names %in% names(dict_name)
10  new_fdata_names[idx] <- dict_name[new_fdata_names[idx]]
11  rownames(fData(eset_pre_c)) <- new_fdata_names
12
13  # For rownames in correlation matrix
14  new_row_names <- rownames(eset_pre_c@assayData$corr)
15  idx <- new_row_names %in% names(dict_name)
16  new_row_names[idx] <- dict_name[new_row_names[idx]]
17  rownames(eset_pre_c@assayData$corr) <- new_row_names
18
19  # For colnames in correlations matrix
20  new_col_names <- colnames(eset_pre_c@assayData$corr)
21  idx <- new_col_names %in% names(dict_name)
22  new_col_names[idx] <- dict_name[new_col_names[idx]]
23  colnames(eset_pre_c@assayData$corr) <- new_col_names
24 }
25

```



```

26 #Plot
27 plotCorrelation(eset_pre_c, type = "circos")

```

8.2 PCA

```

1 #Trying with all T
2 pca_pre <- pca(eset_pre_ts, pca = T)
3 plotPCA(pca_pre, set = "all")
4
5 # Biplot
6 plotPCA(pca_pre, set = "exposures", show.exposures = FALSE) +
7   theme(legend.position = "right") +
8   theme_minimal()
9
10 #Plot the exposures
11 plotEXP(pca_pre) +
12   ggplot2::scale_fill_gradient2(low= "steelblue",
13                                 mid = "white", high = "red") +
14   scale_y_discrete(labels = pretty_label) +
15   labs(y = "Exposures", x="PCs") +
16   geom_point(size = 5) + # increase bullet (point) size
17   theme(
18     text = element_text(size = 18), # overall text size
19     axis.title = element_text(size = 18), # axis labels
20     axis.text = element_text(size = 14), # axis tick labels
21     legend.title = element_text(size = 16), # legend title
22     legend.text = element_text(size = 14) # legend labels
23   ) +
24   guides(colour = guide_legend(override.aes = list(size = 6)))
25
26 #Plot the phenotype
27 plotPHE(pca_pre) + labs(y = "Phenotype", x="PCs")

```

8.3 ExWAS

```

1 ex_cum_pre <- exwas(eset_pre_ts, formula = sLRTI ~ sex + zscore_bweight,
2   family = "gaussian", warnings = F)
3 ex_cum_pre
4 plotExwas(ex_cum_pre) + ggplot2::geom_point(size = 3) + scale_color_
5   manual(values = exposure_colors)
6 plotEffect(ex_cum_pre)
7
8 # Write the result
9 ex_pre_res <- as.data.frame(ex_cum_pre@comparison)
10 rownames(ex_pre_res) <- pretty_label(rownames(ex_pre_res))
11 ex_pre_res <- rownames_to_column(ex_pre_res, var="variable")
12 ex_pre_res <- merge(ex_pre_res, fam, by="variable")
13 write.csv(ex_pre_res, "Exwas_pre_res.csv")
14
15 # Visualisations
16 # Mark the variables as significant
17 exwas_pre <- as.data.frame(ex_cum_pre@comparison) %>%
18   mutate(label = dplyr::recode(rownames(ex_cum_pre@comparison),
19     hresideschild = "Number of children",

```

```

18     unhealthy_diet = "Unhealthy Diet",
19     BPA = "BPA",
20     seifa_disad = "SEIFA disadvantages",
21     GlycA_a = "GlycA (pregnancy)",
22     pregalc = "Prenatal Alcohol Intake",
23     healthy_diet = "Healthy Diet"),
24     sig = pvalue < 0.05)
25
26 # Volcano plot
27 #plotVolcano(ex_cum_pre) +
28 ggplot(exwas_pre, aes(x=as.numeric(effect), y= -log10(as.numeric(pvalue))
29     ,
30     colour = sig)) +
31     geom_point() +
32     labs(x = "Effect", y = expression(-log[10](italic(P)))) +
33     # Create the threshold p-value
34     geom_hline(yintercept = -log10(1.46e-03),
35     col="darkred", linetype = "dashed") +
36     geom_hline(yintercept = -log10(0.05),
37     col="darkgreen", linetype = "dashed") +
38     geom_vline(xintercept = 0, col="blue", linetype = "dashed") +
39     scale_colour_manual(values = c(`FALSE` = "black",
40     `TRUE` = "red")) +
41     coord_cartesian(xlim = c(-5, 5)) +
42     guides(color = "none", shape = "none") +
43     theme_minimal() +
44     theme(panel.border = element_rect(colour = "black",
45     linewidth = 0.8, fill = NA),
46     panel.background = element_rect(fill = "white",
47     color = NA),
48     plot.background = element_rect(fill = "white",
49     color = NA),
50     legend.position = "none") +
51     ggrepel::geom_text_repel(data = subset(exwas_pre, sig),
52     aes(label = label),
53     size = 4,
54     box.padding = unit(0.35, "lines"),
55     point.padding = unit(0.35, "lines"),
56     max.overlaps = Inf,
57     segment.color = "black") +
58     coord_cartesian(xlim = c(-0.5, 0.5)) +
59     labs(x="Effect")

```

8.4 InExWAS

```

1 inexas_cum_pre <- invExWAS(eset_pre_ts, formula = ~zscore_bweight + sex)
2 #head(extract(inexas_cum_pre))
3
4 # Write the result
5 inexas_pre_res <- as.data.frame(inexas_cum_pre@comparison)
6 rownames(inexas_pre_res) <- pretty_label(rownames(inexas_pre_res))
7 inexas_pre_res <- rownames_to_column(inexas_pre_res, var="variable")
8 inexas_pre_res <- merge(inexas_pre_res, fam, by="variable")
9 write.csv(inexas_pre_res, "inExwas_pre_res.csv")
10

```

```

11 # The visualisation
12 plotExwas(inexwas_cum_pre) + geom_point(size=3) + scale_color_manual(
    values = exposure_colors)
13
14 # Create volcano plot
15 inexwas_pre <- as.data.frame(inexwas_cum_pre@comparison) %>%
16   mutate(label = dplyr::recode(rownames(inexwas_cum_pre@comparison),
17     hresideschild = "Number of children",
18     unhealthy_diet = "Unhealthy Diet",
19     BPA = "BPA",
20     seifa_disad = "SEIFA disadvantages",
21     GlycA_a = "GlycA (pregnancy)",
22     sig = pvalue < 0.05)
23
24 ggplot(inexwas_pre, aes(x=effect, y=-log10(pvalue),
25   colour = sig)) +
26   # Create the threshold p-value
27   geom_hline(yintercept = -log10(1.46e-03),
28     col="darkred", linetype = "dashed") +
29   geom_hline(yintercept = -log10(0.05),
30     col="darkgreen", linetype = "dashed") +
31   geom_vline(xintercept = 0, col="blue", linetype = "dashed") +
32   geom_point() +
33   # Create the label
34   labs(x = "Effect", y = expression(-log[10](P-value))) +
35   scale_colour_manual(values = c(`FALSE` = "black",
36     `TRUE` = "red")) +
37   coord_cartesian(xlim = c(-0.3, 0.3)) +
38   guides(color = "none", shape = "none") +
39   ggrepel::geom_text_repel(data = subset(inexwas_pre, sig),
40     aes(label = label),
41     size = 4,
42     box.padding = unit(0.5, "lines"),
43     point.padding = unit(0.35, "lines"),
44     max.overlaps = Inf,
45     segment.color = "black",
46     min.segment.length = 0) +
47   theme_minimal() +
48   theme(panel.border = element_rect(colour = "black",
49     linewidth = 0.8,
50     fill = NA),
51     panel.background = element_rect(fill = "white",
52     color = NA),
53     plot.background = element_rect(fill = "white",
54     color = NA))
55
56 # Write the result
57 write.csv(inexwas_cum_pre@comparison, "Result_inexwas_prenatal.csv")

```

8.5 MExWAS

Understanding elastic net:

- If coefficient > 0: higher values of that variable are associated with an increased value of the outcome

- If coefficient < 0: higher values of the predictor are associated with a decrease in the outcome.

```

1 #Get the data
2 pre_vars_st <- expos(eset_pre_ts)
3
4 #glmnet
5 cv_pre <- cv.glmnet(as.matrix(pre_vars_st), Y,
6                     family = "gaussian", alpha = 0.5)
7 enet_pre_cum <- glmnet(pre_vars_st, Y, alpha = 0.5, lambda = cv_pre$
8                       lambda.min)
9
10 # Plot the graph
11 coef(enet_pre_cum, s = cv_pre$lambda.min) %>%
12   as.matrix() %>% as.data.frame() %>%
13   setNames("coef") %>%
14   rownames_to_column("feature") %>%
15   filter(feature != "(Intercept)") %>%
16   ggplot(aes(x=feature, y=coef, fill=coef)) +
17   scale_x_discrete(labels=pretty_label) +
18   geom_col() +
19   scale_fill_gradient2(low= "steelblue", mid = "white",
20                       high = "red") +
21   coord_flip() +
22   labs(
23     x = NULL, y = "Coefficient",
24     fill = "Value"
25   ) +
26   theme_minimal(base_size = 12)

```

9 Sensitivity Analyses

The biomarkers and MGM can be a confounding factors between the exposures and the outcome, we want to know if the analysis is robust or not.

```

1 # Update the variables
2 desc_noox <- desc_new %>%
3   filter(!Family %in% c("Biomarker", "MGM")) %>%
4   filter(!(Name %in% c("mage")))

```

9.1 Postnatal

9.1.1 Pre-processing ExwAS

```

1 #Select the postnatal variable
2 vars_post_cov <- desc_noox %>%
3   filter(Status == "postnatal") %>%
4   pull(Name)
5 vars_post_cov <- c("bisid", vars_post_cov)
6
7 #Write the variables documents
8 post_vars_cov <- finalvars[,vars_post_cov]
9 post_vars_cov[] <- lapply(post_vars_cov, as.numeric)
10 write.csv(post_vars_cov, "exposome_post_vars_cov_vs.csv", row.names = F)

```

```

11
12 #Description for postnatal
13 vars_post_cov <- vars_post_cov[vars_post_cov != "bisid"]
14 desc_post_cov <- desc_noox[desc_noox$Name %in% vars_post_cov,]
15 write.csv(desc_post_cov, "description_update_post_cov.csv")
16
17 final_vars_post_cov_path <- "/Users/victoriasjalimcopy/Documents/BIOX7011
    /Coding/BIOX7011/Final/exposome_post_vars_cov_vs.csv"
18
19 final_desc_post_cov_path <- "/Users/victoriasjalimcopy/Documents/BIOX7011
    /Coding/BIOX7011/Final/description_update_post_cov.csv"
20
21 #using the rexposome
22 eset_post_cov <- readExposome(
23   exposures = final_vars_post_cov_path,
24   phenotype = final_out_cov_path,
25   description = final_desc_post_cov_path,
26   description.famCol = "Family",
27   description.expCol = "Name",
28   exposures.samCol = "bisid",
29   phenotype.samCol = "bisid",
30   exposures.asFactor = FALSE)
31 eset_post_cov
32
33 # Transform the continuous data
34 eset_post_t_cov <- trans(eset_post_cov, fun = expo_best_trans)
35 #Standardize
36 eset_post_ts_cov <- standardize(eset_post_t_cov, method = "normal")
37 eset_post_ts
38
39 # Summary Statistics
40 plotFamily(eset_post_ts_cov, family = "all")
41
42 # EWAS
43 ex_cum_pos_cov <- exwas(eset_post_ts_cov, formula = cumwhecgh ~ sex +
    zscore_bweight, family = "gaussian", warnings = F)
44 ex_cum_pos_cov
45 plotExwas(ex_cum_pos_cov)
46 plotEffect(ex_cum_pos_cov)
47
48 exwas_pos_cov <- as.data.frame(ex_cum_pos_cov@comparison) %>%
49   mutate(label = dplyr::recode(rownames(ex_cum_pos_cov@comparison),
50     GlycA12m = "GlycA (12m)",
51     financialprob = "Financial Problem",
52     fumany = "Any fumigations",
53     ms smoke_m12 = "Mother smoke (12m)"),
54     sig = pvalue < 0.05)
55
56 # Volcano plot
57 #plotVolcano(ex_cum_pre) +
58 ex_post_sens <- ggplot(exwas_pos_cov, aes(x=as.numeric(effect), y= -log10
    (as.numeric(pvalue))),
59   colour = sig)) +
60   geom_point() +
61   labs(x = "Effect", y = expression(-log[10](italic(P)))) +
62   # Create the threshold p-value

```

```

63 geom_hline(yintercept = -log10(1.46e-03),
64           col="darkred", linetype = "dashed") +
65 geom_hline(yintercept = -log10(0.05),
66           col="darkgreen", linetype = "dashed") +
67 geom_vline(xintercept = 0, col="blue", linetype = "dashed") +
68 scale_colour_manual(values = c(`FALSE` = "black",
69                               `TRUE` = "red")) +
70 coord_cartesian(xlim = c(-5, 5)) +
71 guides(color = "none", shape = "none") +
72 theme_minimal() +
73 theme(panel.border = element_rect(colour = "black",
74                                   linewidth = 0.8, fill = NA),
75       panel.background = element_rect(fill = "white",
76                                       color = NA),
77       plot.background = element_rect(fill = "white",
78                                       color = NA),
79       legend.position = "none") +
80 ggrepel::geom_text_repel(data = subset(exwas_pos_cov, sig),
81                          aes(label = label),
82                          size = 4,
83                          box.padding = unit(0.35, "lines"),
84                          point.padding = unit(0.35, "lines"),
85                          max.overlaps = Inf,
86                          segment.color = "black") +
87 coord_cartesian(xlim = c(-0.5, 0.5)) +
88 labs(x="Effect")
89
90
91 ggsave("post_exwas_sens.png", ex_post_sens, width = 8, height = 6, dpi =
300)

```

9.1.2 InExWAS

```

1 # Inexwas
2 inexwas_cum_post_c <- invExWAS(eset_post_ts_cov, formula = ~zscore_
   bweight + sex)
3
4 #head(extract(inexwas_cum_pre))
5 inexwas_pos_cov <- as.data.frame(inexwas_cum_post_c@comparison) %>%
6   mutate(label = dplyr::recode(rownames(inexwas_cum_post_c@comparison),
7                                pss_m12 = "PSS (12m)",
8                                financialprob = "Financial Problem",
9                                stolenval = "Robbery",
10                               fumany = "Any fumigations",
11                               msmoke_m12 = "Mother smoke (12m)",
12                               buf_500 = "Green space",
13                               pm25_12m = "PM2.5",
14                               drugprob = "Drugs problem",
15                               repel = "Repellent"),
16          sig = pvalue < 0.05)
17
18 # Volcano plot
19 post_inexwas_sens <- ggplot(inexwas_pos_cov, aes(x=as.numeric(effect), y=
   -log10(as.numeric(pvalue))),
20                               colour = sig)) +

```

```

21 geom_point() +
22 labs(x = "Effect", y = expression(-log[10](italic(P)))) +
23 # Create the threshold p-value
24 geom_hline(yintercept = -log10(1.46e-03),
25           col="darkred", linetype = "dashed") +
26 geom_hline(yintercept = -log10(0.05),
27           col="darkgreen", linetype = "dashed") +
28 geom_vline(xintercept = 0, col="blue", linetype = "dashed") +
29 scale_colour_manual(values = c(`FALSE` = "black",
30                               `TRUE` = "red")) +
31 coord_cartesian(xlim = c(-5, 5)) +
32 guides(color = "none", shape = "none") +
33 theme_minimal() +
34 theme(panel.border = element_rect(colour = "black",
35                                   linewidth = 0.8, fill = NA),
36       panel.background = element_rect(fill = "white",
37                                       color = NA),
38       plot.background = element_rect(fill = "white",
39                                       color = NA),
40       legend.position = "none") +
41 ggrepel::geom_text_repel(data = subset(inexwas_pos_cov, sig),
42                          aes(label = label),
43                          size = 4,
44                          box.padding = unit(0.35, "lines"),
45                          point.padding = unit(0.35, "lines"),
46                          max.overlaps = Inf,
47                          segment.color = "black") +
48 coord_cartesian(xlim = c(-0.5, 0.5)) +
49 labs(x="Effect")
50
51 ggsave("post_inexwas_sens.png", post_inexwas_sens, width = 8, height = 9,
52       dpi = 300)

```

9.1.3 MExWAS

```

1 X_post_red <- expos(eset_post_ts_cov)
2
3 #glmnet
4 cv_postcovred <- cv.glmnet(as.matrix(X_post_red), Y, family = "gaussian",
5                          alpha = 0.5)
6
7 #glmnet result
8 post_mexwas_sens <- coef(enet_postcovred, s = cv_postcovred$lambda.min)
9 %>%
10 as.matrix() %>% as.data.frame() %>%
11 setNames("coef") %>%
12 rownames_to_column("feature") %>%
13 filter(feature != "(Intercept)") %>%
14 ggplot(aes(x=feature, y=coef, fill=coef)) +
15 geom_col() +
16 scale_fill_gradient2(low= "steelblue", mid = "white",
17                     high = "red", midpoint=0) +
18 scale_x_discrete(label = pretty_label) +

```

```

18 coord_flip() +
19 labs(
20   x = NULL, y = "Coefficient",
21   fill = "Value"
22 ) +
23 theme_minimal(base_size = 12)
24
25 ggsave("post_mexwas_sens.png", post_mexwas_sens, width = 8, height = 9,
26        dpi = 300)

```

9.2 Prenatal

9.2.1 Data pre-processing ExWAS

```

1 vars_pre_cov <- desc_noox %>%
2   filter(Status == "prenatal") %>%
3   pull(Name)
4 vars_pre_cov <- c("bisid", vars_pre_cov)
5
6 #Write the variables documents
7 pre_vars_cov <- finalvars[,vars_pre_cov]
8 pre_vars_cov[] <- lapply(pre_vars_cov, as.numeric)
9 write.csv(pre_vars_cov, "exposome_pre_vars_cov_vs.csv", row.names = F)
10
11 #Description for postnatal
12 vars_pre_cov <- vars_pre_cov[vars_pre_cov != "bisid"]
13 desc_pre_cov <- desc_noox[desc_noox$Name %in% vars_pre_cov,]
14 write.csv(desc_pre_cov, "description_update_pre_cov.csv")
15
16 final_vars_pre_cov_path <- "/Users/victoriasjalimcopy/Documents/BIOX7011/
17   Coding/BIOX7011/Final/exposome_pre_vars_cov_vs.csv"
18
19 final_desc_pre_cov_path <- "/Users/victoriasjalimcopy/Documents/BIOX7011/
20   Coding/BIOX7011/Final/description_update_pre_cov.csv"
21
22 #using the rexposome
23 eset_pre_cov <- readExposome(
24   exposures = final_vars_pre_cov_path,
25   phenotype = final_out_cov_path,
26   description = final_desc_pre_cov_path,
27   description.famCol = "Family",
28   description.expCol = "Name",
29   exposures.samCol = "bisid",
30   phenotype.samCol = "bisid",
31   exposures.asFactor = FALSE)
32 eset_pre_cov
33
34 # Transform the continuous data
35 eset_pre_t_cov <- trans(eset_pre_cov, fun = expo_best_trans)
36 #Standardize
37 eset_pre_ts_cov <- standardize(eset_pre_t_cov, method = "normal")
38 eset_pre_ts
39
40 # Summary Statistics
41 plotFamily(eset_pre_ts_cov, family = "all")

```



```

40
41 # EWAS
42 ex_cum_pre_cov <- exwas(eset_pre_ts_cov, formula = cumwhecgh ~ sex +
    zscore_bweight, family = "gaussian", warnings = F)
43 ex_cum_pre_cov
44 plotExwas(ex_cum_pre_cov)
45 plotEffect(ex_cum_pre_cov)
46
47 #head(extract(inexwas_cum_pre))
48 exwas_pre_sens <- as.data.frame(ex_cum_pre_cov@comparison) %>%
49   mutate(label = dplyr::recode(rownames(ex_cum_pre_cov@comparison),
50     hresideschild = "Number of children",
51     unhealthy_diet = "Unhealthy Diet",
52     BPA = "BPA",
53     seifa_disad = "SEIFA disadvantages",
54     GlycA_a = "GlycA (pregnancy)",
55     pregalc = "Prenatal Alcohol Intake",
56     healthy_diet = "Healthy Diet"),
57     sig = pvalue < 0.05)
58
59 # Volcano plot
60 exwas_pre_sens <- ggplot(exwas_pre_sens, aes(x=as.numeric(effect), y= -
    log10(as.numeric(pvalue)),
61     colour = sig)) +
62   geom_point() +
63   labs(x = "Effect", y = expression(-log[10](italic(P)))) +
64   # Create the threshold p-value
65   geom_hline(yintercept = -log10(1.46e-03),
66     col="darkred", linetype = "dashed") +
67   geom_hline(yintercept = -log10(0.05),
68     col="darkgreen", linetype = "dashed") +
69   geom_vline(xintercept = 0, col="blue", linetype = "dashed") +
70   scale_colour_manual(values = c(`FALSE` = "black",
71     `TRUE` = "red")) +
72   coord_cartesian(xlim = c(-5, 5)) +
73   guides(color = "none", shape = "none") +
74   theme_minimal() +
75   theme(panel.border = element_rect(colour = "black",
76     linewidth = 0.8, fill = NA),
77     panel.background = element_rect(fill = "white",
78     color = NA),
79     plot.background = element_rect(fill = "white",
80     color = NA),
81     legend.position = "none") +
82   ggrepel::geom_text_repel(data = subset(exwas_pre_sens, sig),
83     aes(label = label),
84     size = 4,
85     box.padding = unit(0.35, "lines"),
86     point.padding = unit(0.35, "lines"),
87     max.overlaps = Inf,
88     segment.color = "black") +
89   coord_cartesian(xlim = c(-0.5, 0.5)) +
90   labs(x="Effect")
91
92 ggsave("pre_exwas_sens.png", exwas_pre_sens, width = 8, height = 6, dpi =
    300)

```

9.2.2 InExWAS

```
1 # Inexwas
2 inexwas_cum_pre_c <- invExWAS(eset_pre_ts_cov, formula = ~sex + zscore_
   bweight)
3 #head(extract(inexwas_cum_pre_c))
4
5 inexwas_pre_sens <- as.data.frame(inexwas_cum_pre_c@comparison) %>%
6   mutate(label = dplyr::recode(rownames(inexwas_cum_pre_c@comparison),
7     hresideschild = "Number of children",
8     unhealthy_diet = "Unhealthy Diet",
9     BPA = "BPA",
10    seifa_disad = "SEIFA disadvantages",
11    GlycA_a = "GlycA (pregnancy)",
12    pregalc = "Pregnancy Alcohol Intake",
13    healthy_diet = "Healthy Diet"),
14     sig = pvalue < 0.05)
15
16 # Volcano plot
17 inex_pre_sens <- ggplot(inexwas_pre_sens, aes(x=as.numeric(effect), y= -
   log10(as.numeric(pvalue))),
18     colour = sig)) +
19   geom_point() +
20   labs(x = "Effect", y = expression(-log[10](italic(P)))) +
21   # Create the threshold p-value
22   geom_hline(yintercept = -log10(1.46e-03),
23     col="darkred", linetype = "dashed") +
24   geom_hline(yintercept = -log10(0.05),
25     col="darkgreen", linetype = "dashed") +
26   geom_vline(xintercept = 0, col="blue", linetype = "dashed") +
27   scale_colour_manual(values = c(`FALSE` = "black",
28     `TRUE` = "red")) +
29   coord_cartesian(xlim = c(-5, 5)) +
30   guides(color = "none", shape = "none") +
31   theme_minimal() +
32   theme(panel.border = element_rect(colour = "black",
33     linewidth = 0.8, fill = NA),
34     panel.background = element_rect(fill = "white",
35     color = NA),
36     plot.background = element_rect(fill = "white",
37     color = NA),
38     legend.position = "none") +
39   ggrepel::geom_text_repel(data = subset(inexwas_pre_sens, sig),
40     aes(label = label),
41     size = 4,
42     box.padding = unit(0.35, "lines"),
43     point.padding = unit(0.35, "lines"),
44     max.overlaps = Inf,
45     segment.color = "black") +
46   coord_cartesian(xlim = c(-0.5, 0.5)) +
47   labs(x="Effect")
48
49 ggsave("pre_inexwas_sens.png", inex_pre_sens, width = 8, height = 10, dpi
   = 300)
```

9.2.3 MExWAS

```
1 X_pre_red <- expos(eset_pre_ts_cov)
2
3 #glmnet
4 cv_precovered <- cv.glmnet(as.matrix(X_pre_red), Y, family = "gaussian",
5   alpha = 0.5)
6 enet_precovered <- glmnet(as.matrix(X_pre_red), Y, alpha = 0.5, lambda =
7   cv_precovered$lambda.min)
8
9 #glmnet result
10 pre_mexwas_sens <- coef(enet_precovered, s = cv_precovered$lambda.min) %>%
11   as.matrix() %>% as.data.frame() %>%
12   setNames("coef") %>%
13   rownames_to_column("feature") %>%
14   filter(feature != "(Intercept)") %>%
15   ggplot(aes(x=feature, y=coef, fill=coef)) +
16   geom_col() +
17   scale_x_discrete(label = pretty_label) +
18   scale_fill_gradient2(low= "steelblue", mid = "white",
19     high = "red", midpoint=0) +
20   coord_flip() +
21   labs(
22     x = NULL, y = "Coefficient",
23     fill = "Value"
24   ) +
25   theme_minimal(base_size = 12)
26
27 ggsave("pre_mexwas_sens.png", pre_mexwas_sens, width = 8, height = 9, dpi
28   = 300)
```

9.3 MExWAS with covariates

Combining the list with the covariates

```
1 evry <- cbind(pre_vars_st, post_vars_st, out_data)
```

9.3.1 Postnatal

```
1 post_cov1 <- evry %>%
2   select(sex, zscore_bweight, colnames(post_vars_st))
3
4 #glmnet
5 cv_postcov <- cv.glmnet(as.matrix(post_cov1), evry$cumwhecgh, family = "
6   gaussian", alpha = 0.5)
7 enet_postcov <- glmnet(as.matrix(post_cov1), evry$cumwhecgh, alpha = 0.5,
8   lambda = cv_postcov$lambda.min)
9
10 #glmnet result
11 wcov_post <- coef(enet_postcov, s = cv_postcov$lambda.min) %>%
12   as.matrix() %>% as.data.frame() %>%
13   setNames("coef") %>%
14   rownames_to_column("feature") %>%
15   filter(feature != "(Intercept)") %>%
```

```

14 ggplot(aes(x=feature, y=coef, fill=coef)) +
15 geom_col() +
16 scale_x_discrete(label = pretty_label) +
17 scale_fill_gradient2(low= "steelblue", mid = "white",
18                      high = "red", midpoint=0) +
19 coord_flip() +
20 labs(
21   x = NULL, y = "Coefficient",
22   fill = "Value"
23 ) +
24 theme_minimal(base_size = 12)
25
26 ggsave("wcov_post_mexwas.png", wcov_post, width = 8, height = 9, dpi =
    300)

```

9.3.2 Prenatal

```

1 pre_cov1 <- evry %>%
2   dplyr::select(sex, zscore_bweight, colnames(pre_vars_st))
3
4 #glmnet
5 cv_precov <- cv.glmnet(as.matrix(pre_cov1), evry$cumwhecgh, family = "
   gaussian", alpha = 0.5)
6 enet_precov <- glmnet(as.matrix(pre_cov1), evry$cumwhecgh, alpha = 0.5,
   lambda = cv_precov$lambda.min)
7
8 #glmnet result
9 wcov_pre <- coef(enet_precov, s = cv_precov$lambda.min) %>%
10  as.matrix() %>% as.data.frame() %>%
11  setNames("coef") %>%
12  rownames_to_column("feature") %>%
13  filter(feature != "(Intercept)") %>%
14  ggplot(aes(x=feature, y=coef, fill=coef)) +
15  geom_col() +
16  scale_fill_gradient2(low= "steelblue", mid = "white",
17                      high = "red", midpoint=0) +
18  scale_x_discrete(label = pretty_label) +
19  coord_flip() +
20  labs(
21   x = NULL, y = "Coefficient",
22   fill = "Value"
23 ) +
24  theme_minimal(base_size = 12)
25
26 ggsave("wcov_pre_mexwas.png", wcov_pre, width = 8, height = 9, dpi = 300)

```

10 DSA

Deletion substitution addition algorithm. Based on many papers, especially from HELIX study, they used this method for multivariable exposure analysis. This actually similar to elastic net that created for variable selection for model at the end.

This one is too long, this is just an example. Would do the rest inside a cluster

We are doing the cross validations first - inside the cluster This is for running the scripts (.sh) format

```
1 #!/bin/bash
2 #SBATCH --nodes=1
3 #SBATCH --ntasks-per-node=1
4 #SBATCH --cpus-per-task=32
5 #SBATCH --mem=48G
6 #SBATCH --job-name=dsa
7 #SBATCH --partition=general
8 #SBATCH --account=a_mcræe
9 #SBATCH --output=slurm_%j.out
10 #SBATCH --error=slurm_%j.err
11 #SBATCH --time=2:00:00
12
13 #Load necessary module
14 module load r/4.4.2
15
16 # For the packages
17 # Set R to use your local library
18 export R_LIBS_USER="/home/uqvsjali/R/x86_64-pc-linux-gnu-library/4.4"
19
20 echo "Using R_LIBS_USER=$R_LIBS_USER"
21 echo "Installed packages:"
22 ls "$R_LIBS_USER"
23 echo "Starting the analysis for postnatal"
24
25 #Running the Rscript
26 Rscript --vanilla "/home/uqvsjali/dsa/dsa.R"
27 echo 'R_LIBS_USER="/home/uqvsjali/R/x86_64-pc-linux-gnu-library/4.4"' >
    ~/.Renvirom
```

Cross-validations (using R script)

```
1 set.seed(177750)
2
3 # Library
4 library(partDSA)
5 library(caret)
6 library(ggplot2)
7 library(tidyverse)
8 library(parallel) #Make computation time faster
9 library(doParallel)
10
11 # Close any existing clusters
12 try(stopCluster(cl), silent = TRUE)
13 try(stopImplicitCluster(), silent = TRUE)
14
15 # Clear all connections
16 closeAllConnections()
17
18 # setup the parallel computing
19 cl <- makeCluster(8)
20 registerDoParallel(cl)
21
22 #Reading the data
23 #Postnatal
```

```

24 X_post <- read.csv("/home/uqvsjali/dsa/postnatal_expo.csv")
25
26 #Prenatal exposures
27 X_pre <- read.csv("/home/uqvsjali/dsa/prenatal_expo.csv")
28
29 all <- cbind(X_post, X_pre)
30
31 #The outcome
32 finalout <- read.csv("/home/uqvsjali/dsa/outcome_update.csv")
33 Y1 <- finalout$cumwhecgh
34 Y2 <- finalout$cumwhecgh_cat
35
36 #The hyperparameter list
37 hyperpar <- expand.grid(
38   cut.off.growth = c(10,15,20), #max partitions to be explored
39   minsplitt = c(20,30), #min number of observation necessary to split
40   minbuck = c(2,3,5) #min observations in one partitions
41 )
42
43 #Function for the tuning
44 cv_dsa <- function(x, y, formula, params, nfolds = 5) {
45   library(parallel)
46   library(caret)
47
48   n <- nrow(x)
49   folds <- createFolds(1:n, k = nfolds, list = TRUE)
50
51   # Use parallel processing for cross-validation folds
52   cv_errors <- mclapply(1:nfolds, function(i) {
53     train_id <- unlist(folds[-i])
54     test_id <- folds[[i]]
55
56     train_x <- x[train_id, , drop = FALSE]
57     test_x <- x[test_id, , drop = FALSE]
58     train_y <- y[train_id]
59     test_y <- y[test_id]
60
61     control_dsa <- DSA.control(
62       cut.off.growth = params$cut.off.growth,
63       minsplitt = params$minsplitt,
64       minbuck = params$minbuck
65     )
66
67     # Try fitting and prediction
68     result <- tryCatch({
69       model_dsa <- partDSA(x = train_x, y = train_y, control = control_
70 dsa)
71       pred <- predict(model_dsa, newdata = test_x)
72       mean((test_y - pred)^2)
73     }, error = function(e) {
74       warning(paste("Error in fold", i, ":", e$message))
75       NA
76     })
77
78     return(result)

```

```

79   }, mc.cores = 8)
80
81   # Return average error
82   return(mean(unlist(cv_errors), na.rm = TRUE))}
83
84 #Let's do the tuning
85 results <- data.frame(hyperpar, cv_error = NA)
86
87 for(i in 1:nrow(hyperpar)) {
88   cat("Testing parameter set", i, "of", nrow(hyperpar), "\n")
89
90   cv_error <- cv_dsa(
91     x = all,
92     y = Y1,
93     formula = Y1 ~ .,
94     params = hyperpar[i, ],
95     nfolds = 5
96   )
97
98   results$cv_error[i] <- cv_error
99
100  # Print progress
101  cat("Parameters:", toString(hyperpar[i, ]),
102      "CV Error:", round(cv_error, 4), "\n")
103 }
104
105 # Find best parameters
106 best_idx <- which.min(results$cv_error)
107 best_params <- hyperpar[best_idx, ]
108 best_error <- results$cv_error[best_idx]

```

The best parameters were: minsplit = 30, minbuck = 5, cut.off.growth = 10. We will do a 30 bootstrap samples (I tried just using different seed but it didn't work because they chose the same variables)

```

1 all <- cbind(X_post, X_pre)
2 # Create 30 bootstrap samples
3 B <- 30
4 # Create a list
5 runs <- vector("list", B)
6
7 for (i in seq_len(B)) {
8   set.seed(i)
9   # Bootstrapping
10  idx <- sample(nrow(all), replace = TRUE)
11  Xb <- all[idx, , drop = FALSE]
12  Yb <- Y[idx]
13
14  # To know which seeds
15  cat("This is seed", i)
16  # Run the DSA for each seeds
17  fit <- partDSA(
18    x = Xb, y = Yb,
19    control = DSA.control(minsplit = 30, minbuck = 5, cut.off.growth =
20      10)
21  )
22  # For each round put it inside the list

```

```

22 runs[[i]] <- list(seed = i, idx = idx, fit = fit)
23 }

1 #Find the important variables that selected each runs
2 var_importances <- lapply(runs, function(x) x$fit$var.importance)
3
4 var_importance_df <- do.call(rbind, lapply(seq_along(var_importances),
5     function(i) {
6         v <- var_importances[[i]]
7         if (is.null(v)) return(NULL)
8         v_df <- as.data.frame(v)
9         v_df$variable <- rownames(v_df)
10        v_df$i <- i
11        v_df
12    })))
13 # Grep it from the COG
14 var_importance_df$sum = rowSums(var_importance_df[, grep("^COG", names(
15     var_importance_df))])
16
17 var_select <- var_importance_df %>% filter(sum > 0) %>%
18     select(i, variable, sum) %>%
19     group_by(variable) %>%
20     summarize(count = n()) %>%
21     mutate(percent = count/30*100) %>%
22     filter(count>1)
23
24 # Sort it by the most selected
25 var_select <- var_select[order(var_select$percent, decreasing = T), ]
26
27 # Visualisations
28 ggplot(var_select, aes(y=reorder(variable, percent), x=percent)) +
29     geom_col(fill = "steelblue") +
30     scale_y_discrete(label = pretty_label) +
31     labs(
32         x = "Percent selected (%)", y = "Exposure"
33     ) +
34     theme_minimal(base_size = 12) +
35     xlim(0,100)
36
37 # Rename it
38 var_select$variable <- pretty_label(var_select$variable)
39 write.csv(var_select, "DSA_select.csv")

```

11 Associations

11.1 Creating multidatasets

```

1 library(devtools)
2 #devtools::install_github("isglobal-brge/omicRexposome")
3 #omicRexposome is not in CRAN yet
4 library(omicRexposome)
5 library(MultiDataSet)

```



```

6 library(SummarizedExperiment)
7 library(Biobase)
8
9 # Data
10 final_lipid <- read.csv("/Users/victoriasjalimcopy/Documents/BIOX7011/
    Coding/BIOX7011/processed_lipid_id.csv", row.names = 1)
11
12 #Create the assay
13 assay_lipid <- t(as.matrix(final_lipid))
14
15 # Pheno data
16 rownames(out_data1) <- out_data1$id
17 pheno <- AnnotatedDataFrame(data= out_data1,
18     varMetadata = data.frame(
19         labelDescription = c("Sample ID",
20             "Cumulative wheeze cough
21                 Change",
22                 "Cumulative wheeze cough
23                 Category",
24                 "Sex (1=Female, 2=Male)",
25                 "Z-score Body Weight"),
26         row.names = colnames(out_data1)))
27
28 #Feature data - This data consist of the name of our lipid & other info
29 desc_lipid <- read.csv("/Users/victoriasjalimcopy/Documents/BIOX7011/
    Coding/BIOX7011/Final/lipid_desc.csv", row.names = 1)
30
31 desc_lipid <- desc_lipid[rownames(assay_lipid), , drop = FALSE]
32
33 feat_dat <- AnnotatedDataFrame(desc_lipid,
34     varMetadata = data.frame(
35         labelDescription = c(
36             "Lipid Family Classification",
37             "Detailed Lipid Description", "chr", "start", "end"),
38         row.names = colnames(desc_lipid)))
39
40 # Expression set
41 lipid_eset <- ExpressionSet(
42     assayData = assay_lipid,
43     phenoData = pheno,
44     featureData = feat_dat,
45     annotation = "Lipidomics_MS"
46 )
47
48 # Create the multidatasets
49 mds <- createMultiDataSet()
50 mds <- add_eset(mds, lipid_eset,
51     dataset.type = "Lipidomics",
52     overwrite = T)
53
54 # Try combine it with the all data
55 mds <- add_exp(mds, eset_finalp)
56 pData(eset_finalp)$id <- sampleNames(eset_finalp)
57
58 #There is a mismatch ID problem - debugging
59 # can't do further analysis because of this

```

```

60 common_id <- commonIds(mds)
61 length(common_id) #same length
62
63 #Reorder the ID variable
64 # Sort numerically
65 numeric_order <- common_id[order(as.numeric(common_id))]

```

11.2 Associations

```

1 # Associations
2 test_assoc <- association(mds, ~sex + zscore_bweight,
3                           expset = "exposures",
4                           omicset = "Lipidomics")
5
6 hit <- tableHits(test_assoc, th=0.001)
7 lab <- tableLambda(test_assoc)
8 result_ass <- merge(hit, lab, by="exposure") %>%
9   arrange(result_ass)
10
11 #RUN THIS IF HAVE TIME - Adding another SVA analysis (more robust)
12 test_sva <- association(mds, ~sex + zscore_bweight,
13                         expset = "exposures",
14                         omicset = "Lipidomics",
15                         sva = "slow")
16 test_sva@results
17 # Calculate how many have hits
18 hit_sva <- tableHits(test_sva, th=0.001)
19 lab_sva <- tableLambda(test_sva)
20 sva <- merge(hit_sva, lab_sva, by="exposure")
21
22 sva$exposure <- pretty_label(sva$exposure)
23 sva$variable <- sva$exposure
24 sva <- merge(fam, sva, by="variable")
25
26 write.csv(sva, "associations_result.csv")

```

11.2.1 Saving the data

Need to find the positions first to be call out from the analysis

```

1 # Atregs
2 which(names(test_sva) == "atreg12m") #8
3 atregs_ass <- as.data.frame(getAssociation(test_sva, rid = 8))
4 write.csv(atregs_ass, "atregs_ass.csv")
5
6
7 #glyca
8 which(names(test_sva) == "GlycA12m") #43
9 glyca_ass <- as.data.frame(getAssociation(test_sva, rid = 43))
10 write.csv(glyca_ass, "glyca_ass.csv")
11
12 #numchild
13 which(names(test_sva) == "hresideschild") #49
14 numchild_ass <- as.data.frame(getAssociation(test_sva, rid = 49))

```

```

15 write.csv(numchild_ass, "numchild_ass.csv")
16
17 #pss
18 which(names(test_sva) == "pss_m12") #78
19 pss_ass <- as.data.frame(getAssociation(test_sva, rid = 78))
20 write.csv(pss_ass, "pss_ass.csv")

```

11.2.2 Plotting the graph

```

1 pval <- -log10(0.001)
2 #plot the protein
3 gridExtra::grid.arrange(
4   plotAssociation(test_sva, rid="GlycA12m", type="protein") +
5   ggplot2::ggtitle("GlycA (12m)") +
6   ggplot2::geom_hline(yintercept = pval, color = "LightPink"),
7   plotAssociation(test_sva, rid="atreg12m", type="protein") +
8   ggplot2::ggtitle("aTregs (12m)") +
9   ggplot2::geom_hline(yintercept = pval, color = "LightPink"),
10  plotAssociation(test_sva, rid="hresideschild", type="protein") +
11  ggplot2::ggtitle("Number of Children") +
12  ggplot2::geom_hline(yintercept = pval, color = "LightPink"),
13  plotAssociation(test_sva, rid="pss_m12", type="protein") +
14  ggplot2::ggtitle("PSS Score (12m)") +
15  ggplot2::geom_hline(yintercept = pval, color = "LightPink"),
16  ncol=3)
17
18 # Plot the volcano
19 vol_ass <- gridExtra::grid.arrange(
20   plotAssociation(test_sva, rid="GlycA12m", type="volcano", tPV=pval)+
21   ggplot2::ggtitle("GlycA \n at 12m") +
22   ggplot2::theme(plot.title = element_text(hjust = 0.5)) +
23   ggplot2::ylim(0,25),
24   plotAssociation(test_sva, rid="atreg12m", type="volcano", tPV=pval)+
25   ggplot2::ggtitle("aTregs \n at 12m") +
26   ggplot2::theme(plot.title = element_text(hjust = 0.5)) +
27   ggplot2::ylim(0, 25),
28   plotAssociation(test_sva, rid="hresideschild", type="volcano", tPV=
pval) +
29   ggplot2::ggtitle("Number of \n Children") +
30   ggplot2::theme(plot.title = element_text(hjust = 0.5)) +
31   ggplot2::ylim(0, 25),
32   plotAssociation(test_sva, rid="pss_m12", type="volcano", tPV=pval) +
33   ggplot2::ggtitle("PSS Score \n at 12m") +
34   ggplot2::theme(plot.title = element_text(hjust = 0.5)) +
35   ggplot2::ylim(0, 25) +
36   ggplot2::geom_hline(yintercept = -log10(0.001),
37                       colour ="grey", linetype ="twodash"), ncol=4)
38 ggsave(filename = "vol_ass.png", vol_ass, width = 8, height = 9, dpi =
39         300)
40 # Plot qq
41 gridExtra::grid.arrange(
42   plotAssociation(test_sva, rid="GlycA12m", type="qq")+
43   ggplot2::ggtitle("GlycA \n at 12m") +

```

```

44     ggplot2::theme(plot.title = element_text(hjust = 0.5)),
45     plotAssociation(test_sva, rid="atreg12m", type="qq")+
46     ggplot2::ggtitle("aTregs \n at 12m") +
47     ggplot2::theme(plot.title = element_text(hjust = 0.5)),
48     plotAssociation(test_sva, rid="hresideschild", type="qq") +
49     ggplot2::ggtitle("Number of \n Children") +
50     ggplot2::theme(plot.title = element_text(hjust = 0.5)),
51     plotAssociation(test_sva, rid="pss_m12", type="qq") + ggplot2::
ggtitle("PSS Score \n at 12m") +
52     ggplot2::theme(plot.title = element_text(hjust = 0.5)),
53     plotAssociation(test_sva, rid="healthy_diet", type="qq") +
54     ggplot2::ggtitle("Healthy Diet") +
55     ggplot2::theme(plot.title = element_text(hjust = 0.5)), ncol=5)

```

12 Mediation

```

1 library(mediation)
2
3 # Write functions for automatic mediations
4 quick_mediation <- function(data, x, y, mediators, covars = NULL) {
5
6     med_names <- colnames(mediators)
7
8     out <- lapply(med_names, function(med) {
9         cat("\n=== Testing mediator:", med, "===\n")
10        tryCatch({
11            if (is.null(covars)) {
12                # Build formulas as strings first
13                f_m_str <- paste(med, "~", x)
14                f_y_str <- paste(y, "~", med, "+", x)
15            } else {
16                rhs_vars <- paste(c(x, covars), collapse = " + ")
17                f_m_str <- paste(med, "~", rhs_vars)
18                f_y_str <- paste(y, "~", med, "+", rhs_vars)
19            }
20
21            # Convert to formulas and fit models
22            f_m <- as.formula(f_m_str)
23            f_y <- as.formula(f_y_str)
24
25            mod.m <- lm(f_m, data = data)
26            mod.y <- lm(f_y, data = data)
27
28            mod.m$call$formula <- f_m
29            mod.y$call$formula <- f_y
30
31            # Run mediation - this is the key fix
32            fit <- mediation::mediate(
33                model.m = mod.m,
34                model.y = mod.y,
35                treat = x,
36                mediator = med,
37                sims = 500,
38                boot = T

```

```

39   )
40   cat("Mediation completed for", med, "\n")
41
42   # Extract results
43   data.frame(
44     mediator = med,
45     ACME = fit$d0,
46     ACME_CI_L = fit$d0.ci[1],
47     ACME_CI_U = fit$d0.ci[2],
48     ACME_p = fit$d0.p,
49     ADE = fit$z0,
50     ADE_CI_L = fit$z0.ci[1],
51     ADE_CI_U = fit$z0.ci[2],
52     ADE_p = fit$z0.p,
53     TotalEffect = fit$tau.coef,
54     Total_CI_L = fit$tau.ci[1],
55     Total_CI_U = fit$tau.ci[2],
56     Total_p = fit$tau.p,
57     PropMediated = fit$n0,
58     PropMed_CI_L = fit$n0.ci[1],
59     PropMed_CI_U = fit$n0.ci[2],
60     PropMed_p = fit$n0.p,
61     stringsAsFactors = FALSE,
62   )
63
64   }, error = function(e) {
65     # Return empty row if mediation fails
66     cat("Error with mediator", med, ":", e$message, "\n")
67     return(NULL)
68   })
69 })
70
71 # Remove NULL results and bind rows
72 out <- out[!sapply(out, is.null)]
73 if(length(out) > 0) {
74   do.call(rbind, out)
75 } else {
76   cat("No successful mediations\n")
77   return(data.frame())
78 }
79 }
80
81 # GlycA
82 medglyc <- quick_mediation(evry, "GlycA12m", "cumwhecgh", final_lipid,
83   covars = c("sex", "zscore_bweight"))
84 medglyc <- medglyc %>%
85   mutate(p.adj = p.adjust(ADE_p, method = "fdr"))
86 write.csv(medglyc, "Mediations_glyca.csv", row.names = F)
87
88 # atregs
89 medatregs <- quick_mediation(evry, "atreg12m", "cumwhecgh",
90   final_lipid, covars = c("sex", "zscore_
91   bweight"))
92 medatregs <- medatregs %>%
93   mutate(p.adj = p.adjust(ADE_p, method = "fdr"))

```

```

93 write.csv(medatregs, "Mediation_atregs.csv", row.names = F)
94
95 #N children
96 med_hresides <- quick_mediation(evry,"hresideschild", "cumwhecgh", final_
    lipid,covars = c("sex","zscore_bweight"))
97 #Adjusting p-value
98 med_hresides <- med_hresides %>%
99   mutate(p.adj = p.adjust(ADE_p, method = "fdr"))
100 write.csv(med_hresides, "Mediation_nchildren.csv", row.names = F)
101
102 #Diet
103 med_diet <- quick_mediation(evry,"healthy_diet", "cumwhecgh", final_lipid
    ,covars = c("sex","zscore_bweight"))
104 med_diet <- med_diet %>%
105   mutate(p.adj = p.adjust(ADE_p, method = "fdrddad"))
106 write.csv(med_diet, "Mediation_healthydiet.csv", row.names = F)
107
108 # PSS
109 med_pss <- quick_mediation(evry,"pss_m12", "cumwhecgh", final_lipid,
    covars = c("sex","zscore_bweight"))
110 med_pss <- med_pss %>%
111   mutate(p.adj = p.adjust(ADE_p, method = "fdr"))
112 write.csv(med_pss, "Mediation_pss.csv", row.names = F)

```

13 Partial correlations

```

1 # Select the significant data
2 mds_ord@assayData$exposures$exp
3 out_id <- mds_ord@phenoData$exposures$main@data %>%
4   select(id, cumwhecgh)
5
6 all_id <- as.data.frame(t(mds_ord@assayData$exposures$exp))
7 all_id <- rownames_to_column(all_id, var="id")
8
9 all_data_phen <- merge(all_id, out_id, by = "id")
10 sel_sig <- all_data_phen %>%
11   select(hresideschild, pregalc, healthy_diet, GlycA12m, fumany, msmoke_
    m12, pss_m12, cumwhecgh)
12
13 colnames(sel_sig) <- pretty_label(colnames(sel_sig))
14
15 # Visualisations
16 # Heatmap
17 library(corrplot)
18
19 estimate_cor <- all_pcor$est
20 p_cor <- all_pcor$pval
21
22 png("corrplot_pcor.png", width = 9, height = 9, units = "in", res = 300)
23
24 # Hide non-significant cells (FDR ≥ 0.05)
25 cor_all <- corrplot(estimate_cor,
26   method = "color",
27   type = "full",

```

```

28     tl.col = "black",
29     tl.srt = 45,
30     p.mat = p_cor,
31     sig.level = 0.05,
32     insig = "blank",
33     addgrid.col = NA,
34     addCoef.col = "black",
35     number.cex = 0.7,
36     col = colorRampPalette(c("steelblue", "white", "firebrick"))
    (200))
37
38 dev.off()
39
40 corrplot(cor_pht)
41
42 # The network
43 library(ggraph)
44 library(tidygraph)
45
46 # Choose edges (find the significant one)
47 mask <- (p_cor < 0.05)
48 edges <- which(mask & upper.tri(estimate_cor), arr.ind = TRUE)
49
50 edgedf <- data.frame(
51   from = rownames(estimate_cor)[edges[,1]],
52   to   = colnames(estimate_cor)[edges[,2]],
53   r     = estimate_cor[edges]
54 )
55
56 # Build graph
57 g <- graph_from_data_frame(edgedf, directed = FALSE)
58
59 g_tbl <- as_tbl_graph(g)
60
61 ggraph(g_tbl, layout = "fr") +
62   geom_edge_link(aes(width = abs(r), color = r > 0), alpha = 0.6) +
63   geom_node_point(size = 5, color = "grey80") +
64   geom_node_text(aes(label = name), repel = TRUE, size = 3,
65                 point.padding = unit(0.25, "lines"),
66                 box.padding   = unit(0.5, "lines"),
67                 segment.size = 0) +
68   scale_edge_width(range = c(0.4, 3)) +
69   scale_edge_color_manual(values = c("steelblue", "firebrick"),
70                          labels = c("Negative", "Positive")) +
71   theme_void() +
72   theme(legend.position = "bottom")

```